RUANDER
OKTATÓKÖZPONT

Tesztautomatizálás

Botlik Béla

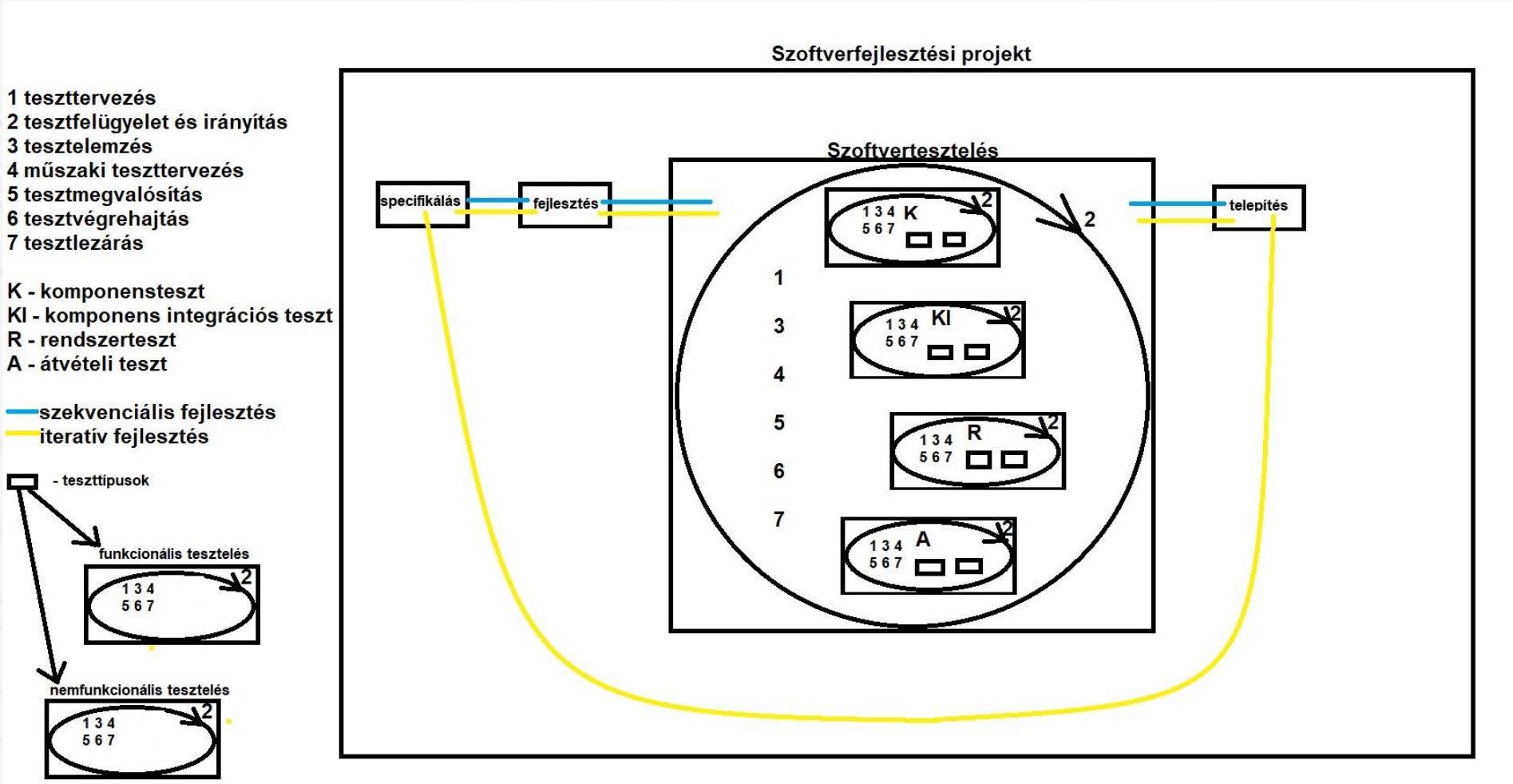
CTFL ismétlés I.

Tesztevékenység	Feladatok	Munkatermékek
Teszttervezés	A teszt céljának, tárgyának, megvalósításának meghatározása.	teszterv, tesztütemterv, kockázati nyilvántartás, belépési és kilépési feltételek
Tesztfelügyelet és -irányítás	A tesztevékenységek ellenőrzése és irányítása.	tesztjelentések, irányelvek, kockázati információk, tesztelőrehaladási jelentések
Tesztelemzés	A tesztbázis elemzése a szoftver tesztelhetőségének feltárására.	tesztfeltételek, tesztbázis hibajelentései
Műszaki teszttervezés	Tesztesetek készítése, tesztkörnyezet tervezése.	tesztesetek, lefedettségi elemek, tesztadatokra és a tesztkörnyezetre vonatkozó követelmények
Tesztmegvalósítás	Teszteljárások, tesztkészletek készítése, tesztkörnyezet megvalósítása.	teszteljárások, tesztkészletek, tesztszkriptek, tesztadatok, végrehajtási ütemterv, tesztkörnyezet
Tesztvégrehajtás	A tesztek futtatása a tesztelési ütemterv szerint.	tesztnaplók, hibajelentések
Tesztlezárás	Tesztjelentések készítése.	összefoglaló tesztjelentés, változtatáskérés

CTFL ismétlés II.

	tesztbázis	tesztcél	teszt tárgya	hibák	megközelítés
komponensteszt	tervek, kód, adatmodell	komp. verifikálás, hibák feltárása, bizalom kivívása	komponens, osztály, adatstruktúra	hibás funkcionalitás, programozói hibák	Kódközeli, tipikusan a fejlesztők végzik.
komponensintegrációs teszt	tervezési diagramok, interfész leírások	kockázat csökkentés, interfész verifikálás, hibák feltárása	interfészek	helytelen/hiányos adatok, hívások helytelen sorrendje, adatok félreértelmezése	Funkcionalitás helyett a kommunikációra fókuszál, a fejlesztők végzik.
rendszerteszt	rendszer/szoftver-követelmények	rendszer verifikálás, bizalom kivívása	alkalmazás, op.rendszer, rendszer	helytelen viselkedés, helytelen vezérlés	Végponttól végpontig terjedő tesztek, független tesztelők is végezhetik.
rendszerintegrációs teszt	rendszerterv, használati esetek	kockázat csökkentés, interfész verifikálás, hibák feltárása	alrendszerek, API-k, külső szolgáltatások	rendszerek közötti kommunikáció hibái, átadott adatok félreértelmezése	Funkcionalitás helyett a kommunikációra fókuszál, a tesztelők végzik.
elfogadási teszt	rendszer/szoftver-követelmények	rendszer validálás, kiadásra való felkészültség igazolása	rendszer	követelményeknek nem megfelelés	A tesztek felelősei üzleti felhasználók, vevők.

CTFL ismétlés III.



Bevezetés és célok a tesztautomatizálás terén (I. fejezet)

- A tesztautomatizálás célja
- A tesztautomatizálás előnyei
- A tesztautomatizálás hátrányai
- A tesztautomatizálás korlátai
- Tesztautomatizálás különböző fejlesztési életciklusmodellekben
- Teszteszköz választás

Tesztautomatizálás

- A tesztautomatizálás tevékenységei:
  - célzott szoftvereszközök használata tesztkészletek előkészítéséhez és kezeléséhez a tesztvégrehajtás érdekében;
  - tesztek automatizált végrehajtása;
  - az aktuális és elvárt eredmények összehasonlítása.
- A tesztautomatizálás széleskörű szoftverterületeket fed le és sokféle szoftverre vonatkozhat
  - UI SUT; SUT UI nélkül; mobilappok; hálózati protokollok és kapcsolatok

A tesztautomatizálás előnyei, hátrányai és korlátai

Előnyök	Hátrányok	Korlátok
Több teszt futtatása	Kezdeti befektetés szükséges	Nem minden teszt automatizálható
Manuálisan végre nem hajtható tesztek futtatása	A tesztautomatizálási megoldás fejlesztése és karbantartása időigényes lehet	Csak azt tudja tesztelni, amire programozták
Összetettebb tesztek elvégzése	Esetleges plusz költségek	Teszt-orákulum hitelesítést igényel
Tesztelési erőforrások hatékony kihasználása	Kevésbé alkalmazkodó a rendszer változásaihoz	Csak géppel értelmezhető teszteredményeket tud ellenőrizni
Kevesebb emberi hiba	Új hibákat vezethet be	
Gyorsabb végrehajtás	Világos tesztcélokra van szükség	
Gyors visszajelzés a minőségről		
Javítja a rendszer megbízhatóságát		
Következetes tesztvégrehajtás		

Tesztautomatizálás és fejlesztési modellek

- A különböző fejlesztési modellek más jellemzőkkel bírnak a tesztautomatizálás szempontjából:
  - Vízesés modell: a tesztautomatizálás a megvalósítással párhuzamosan vagy azt követően zajlik.
  - V-modell: minden fejlesztési szinthez tartozik tesztelési szint.
  - Agilis fejlesztés: gyakori automatizált tesztvégrehajtás jellemzi.
- A legmegfelelőbb teszteszköz kiválasztásához a tesztelt rendszert (SUT) és a csapat összetételét kell kiértékelni.

Felkészülés a tesztautomatizálásra (II. fejezet)

- Tesztelhetőség és konfiguráció
- Tesztautomatizálás végrehajtási környezetei
- A tesztelendő rendszer elemzése
- Eszközértékelés

Tesztelhetőség és konfigurációs megoldások

- Az automatizálásra való felkészülés első lépése a tesztelhetőség meghatározása.
- A jobb tesztelhetőség konfigurációs beállításokat igénylő megoldásai:
  - hozzáférhetőségi azonosítók;
  - rendszerkörnyezeti változók;
  - telepítési változók.
- A tesztelhetőség tervezésének aspektusai:
  - megfigyelhetőség;
  - irányíthatóság;
  - architektúra átláthatósága.

Tesztautomatizálás különböző környezetben

- Automatizált tesztek végrehajtási környezetei:
  - lokális fejlesztési környezet: komponensteszt; fehérdobozos teszt
  - build környezet: CI/CD folyamat; komponensintegrációs teszt
  - integrációs környezet: feketedobozos és rendszer(integrációs) teszt
  - validációs környezet: elfogadási tesztelés
  - működési környezet: speciális tesztek (canary release, blue/green deployment, A/B teszt)

A tesztelendő rendszer elemzése

- Egy sikeres tesztautomatizálási megoldás (TAS) létrehozásának előfeltétele a tesztelendő rendszer (SUT) elemzése.
- A tesztautomatizálási megközelítés és architektúra követelményeinek szempontjai:
  - tesztttevékenységek;
  - tesztszintek;
  - teszttípusok;
  - tesztszerepkörök;
  - szoftvertermékek;
  - SUT és TAS kompatibilitás;
  - tesztadatok elérhetősége és minősége;
  - harmadik féltől származó alkalmazások emulálása.

Feladat I.

- Elemezze a tesztelt rendszert (SUT) és keressen tesztautomatizálási megoldást!
  - A vizsgált rendszer egy webáruház, amely lehetővé teszi a felhasználók számára:
    - Regisztrációt és bejelentkezést
    - Termékek keresését és megtekintését
    - Kosárba helyezést és vásárlást
    - Fizetést PayPal integrációval
  - A rendszer többféle technológiai rétegből áll:
    - Frontend (React)
    - Backend (Java – Spring Boot)
    - Adatbázis (MySql)
    - Külső API (PayPal)

Az eszközértékelés eredményei

- A rendszer elemzéséből táblázatos formában összeáll a használható eszközök köre, legfőbb tulajdonságaikkal:
  - technológia és IDE;
  - konfigurálhatóság;
  - tesztadatok kezelhetősége;
  - teszttípusonkénti eltérések;
  - riportálási képesség;
  - más eszközökkel való integrálhatóság;
  - az architektúra bővítésének lehetősége.

Feladat II.

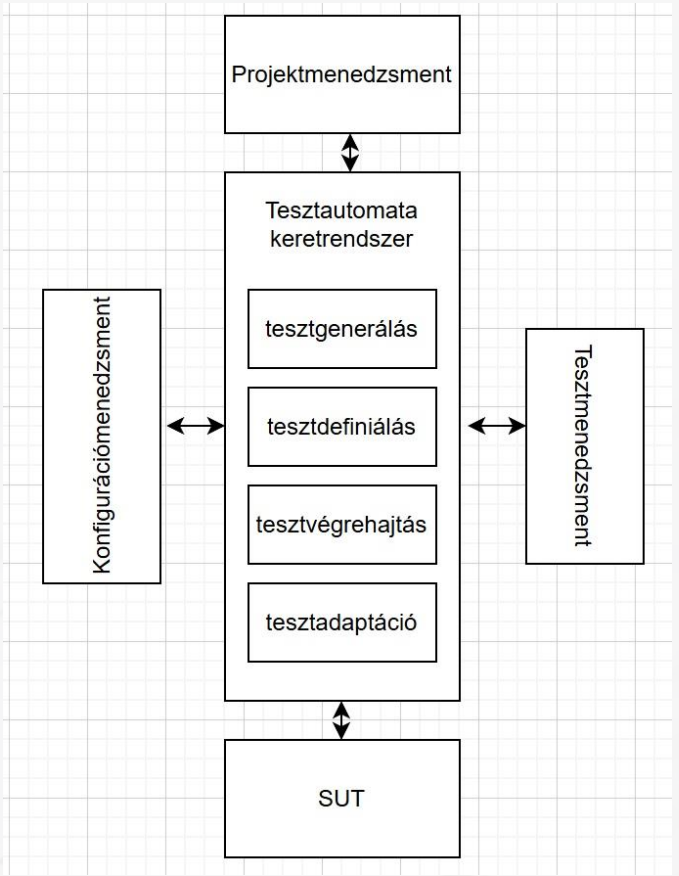
- Az előző feladat webáruházának PayPal API teszteléséhez első körben 3 eszköz merült fel:
  - Postman - Grafikus felhasználói felülettel rendelkező API tesztelő
  - RestAssured - Java alapú API tesztelési könyvtár fejlesztőközpontú megközelítéssel
  - Karate - BDD megközelítésű API tesztelő keretrendszer
- Töltse ki az eszközértékelés eredményeit összehasonlító táblázatot!

Tesztautomatizálási architektúra (III. fejezet)

- Általános tesztautomatizálási architektúra
- Tesztautomatizálási megoldás tervezése
- Tesztautomatizálási keretrendszer (TAF)
- Tesztautomatizálási megközelítések
- Tervezési alapelvek és tervezési minták a tesztautomatizálásban

A tesztautomatizálási architektúra és fő képességei

- A tesztautomatizálási architektúra interfészei:
  - SUT interfész;
  - Projektmenedzsment interfész;
  - Tesztmenedzsment interfész;
  - Konfigurációmenedzsment interfész.
- Az automatizálási keretrendszer képességei:
  - Tesztgenerálás;
  - Tesztdefiniálás;
  - Tesztvégrehajtás;
  - Tesztadaptáció.



Tesztautomatizálási megoldás tervezése

- A tesztautomatizálási megoldás (TAS) tervezése a SUT követelményei és a szükséges eszközök alapján kerül megvalósításra, melynek elemei:
  - Tesztautomata eszközök és eszközspecifikus könyvtárak kiválasztása
  - Beépülő modulok és/vagy komponensek fejlesztése
  - Kapcsolati és interfész követelmények azonosítása
  - Kapcsolódás tesztmenedzsment és hibakezelő eszközökhöz
  - Verziókezelési rendszer és adattárak használata

Tesztautomatizálási keretrendszer

- A TAS technikai alapja a tesztautomatizálási keretrendszer (TAF).

Elemei	Jellemzői	Rétegei
tesztellenőrző rendszer	struktúráltság	tesztszkriptek
tesztkönyvtárak	automatizálási hatékonyság	üzleti logika
tesztszkriptek	skálázhatóság	alapkódkönyvtárak
tesztkészletek		

Feladat III.

- Alkalmazza a tesztautomatizálási keretrendszer rétegezését egy webalkalmazás funkcionális teszteléséhez (belépési funkció)!

Tesztautomatizálási megközelítések I.

- Számos megközelítés létezik a tesztautomatizálás terén, melyek különböző körülmények között alkalmazhatók:

Megközelítés	Főbb jellemzők	Előnyök	Hátrányok
Rögzítés/lejátszás	A manuális tesztelési műveletek elektronikus rögzítésén és visszajátszásán alapul.	Kezdetben könnyen beállítható és használható.	Fenntartása/továbbfejlesztése nehézkes; SUT elérhető kell legyen; Ritkán és kismértékben változó SUT esetén ésszerű; SUT-verziófüggő; A tesztesetek rögzítése időigényes lehet.
Lineáris szkriptelés	A rögzítés/lejátszás tesztjeit felhasználó programozási tevékenység	Gyors és egyszerű; A tesztszkriptek módosíthatók.	Fenntartása/továbbfejlesztése nehézkes; SUT elérhető kell legyen; Ritkán és kismértékben változó SUT esetén ésszerű; Némi programozási ismeret szükséges.

Tesztautomatizálási megközelítések II.

Megközelítés	Főbb jellemzők	Előnyök	Hátrányok
Struktúrált szkriptelés	Blokkokba, könyvtárakba szervezett szkriptek	újrafelhasználható; karbantartható; rugalmas; az üzleti logika elkülöníthető.	programozási ismeret szükséges; nagyobb kezdeti befektetés; komplexitás.
Adatalapú tesztelés (DDT)	Struktúrált szkriptre épülő, adatokkal ellátott szkriptek	tesztesetek egyszerű bővítése; tesztköltségek csökkenése; tesztelemzők nagyfokú szabadsága.	adatkezelési kihívások
Kulcsszóvezérelt tesztelés (KDT)	Gyakran DDT-re épülő, kulcsszavak és tesztadatok alapján származtatott lépések listája vagy táblázata	teszt -és üzleti elemzők részvétele a tesztesetek létrehozásában; manuális tesztekre is alkalmazható; kevésbé függ a fejlesztktől.	A kulcsszavak implementálása és karbantartása összetett feladat; Kis rendszerek esetén jelentős erőforrás igénye van.

Tesztautomatizálási megközelítések III.

Megközelítés	Főbb jellemzők	Előnyök	Hátrányok
TDD (tesztvezérelt fejlesztés)	A tesztek a fejlesztés részeként és elsőként kerülnek meghatározásra	egyszerű teszteset fejlesztés; jobb kódminőség, - tesztelhetőség, - kódlefedettség elérés; csökkenti a hibák tovább terjedését; javuló kommunikáció; gyors kilépési kritérium elérés.	TDD-re áttérés időigényes;
BDD (viselkedésvezérelt fejlesztés)	Természetes nyelvformátum használatával kialakított elfogadási kritériumok, amik egyúttal tesztesetek is	javuló kommunikáció; BDD forgatókönyvek tesztesetként biztosítják a specifikáció lefedettségét; többféle teszttípus előállításában használható a tesztpiramis különböző szintjein.	további teszteseteket kell meghatározni; üzleti elemzők és fejlesztők kihagyása; tesztlépések implementálása bonyolult; emiatt nehézkes és költséges hibakeresés.

Feladat IV.

- Határozzon meg különböző tesztautomatizálási megközelítéseket egy e-kereskedelmi webáruház kosár és fizetés folyamat teszteléséhez!

Tervezési alapelvek és tervezési minták a tesztautomatizálásban

- A szoftverfejlesztő mellett a tesztautomatizálási mérnöknek is ismernie kell a tervezési alapelveket és mintákat:

OOP alapelvek	SOLID alapelvek	Tervezési minták
absztrakció	egyért felelősség	Facade pattern
egységbezárás	nyitottság és zártság	Singleton pattern
adatrejtés	Liskov helyettesítés	POM (Page Object Model)
öröklődés	interfész helyettesítés	Folyamat-alapú modell (FMP)
polimorfizmus	függőség inverzió	

Feladat V.

- Adjon meg néhány tervezési mintát -a tervezési alapelvek betartásával- egy e-kereskedelmi webáruház termék kereső és kosárba helyezés folyamatához!

A tesztautomatizálás megvalósítása (IV. fejezet)

- Pilot projekt
- Folyamat integráció és szervezeti tényezők
- Kockázatelemzés és kockázatcsökkentés a tesztautomatizálásban
- A tesztautomatizálási megoldás fenntarthatósága

A tesztautomatizálási pilot projekt sikerének kulcsténye zői

- A pilot projekt egy rövid, de kulcsfontosságú tesztidőszak a tesztautomatizálási stratégia meghatározásához.
- Az értékelés legfontosabb tényezői:
  - Használt programozási nyelvek és eszközök (kereskedelmi vagy nyílt forráskódú)
  - Lefedendő tesztszintek
  - Kiválasztott tesztesetek
  - Tesztesetek fejlesztési megközelítése

CI/CD integráció és szervezeti tényezők a tesztautomatizálásban

- CI/CD folyamat integrációja:
  - A tesztautomatizálási rendszer (TAS) beillesztése a folyamatos integrációs folyamatokba.
  - A kezdeti CI/CD beállítás módosítása a tesztek növekedésével.
- Egyéb, nem technikai szempontok értékelése:
  - A csapattagok tudása és tapasztalata
  - A csapat szerkezete és együttműködése
  - Licencelési és szervezeti szabályok
  - A tesztelési célok és tesztesetek lefedése

Feladat VI.

- Egy cég úgy döntött, bevezeti a tesztautomatizálást a weboldaluk fizetési folyamatának ellenőrzésére, amit pilot projekttel indítanak. Ehhez Selenium Webdrivert választanak.
  - Alkalmazzon irányelveket, amelyek támogatják a hatékony tesztautomatizálási pilot és bevezetési tevékenységeket!

Telepítési kockázatok elemzése

- TAF és SUT összekapcsolása
- Bevezetési kockázatok
  - Tűzfalak megnyitása
  - Erőforrások kihasználtsága (CPU, RAM)
- Felkészülés és kockázatkezelés
  - Hálózati kapcsolat és megbízhatóság
  - Mobil tesztautomatizálás kihívásai

Tesztautomatizálás és kockázatcsökkentés

- Technikai bevezetési kockázatok
  - Tesztcsomag összeállítása
  - Naplózás és logolási szintek
  - Tesztstruktúra kialakítása
  - Frissítések kezelése
- Kockázatcsökkentési stratégiák
  - Verziókezelés és központi tárolás
  - Megbízható tesztkörnyezet és előfeltételek
  - Tesztek ismételhetősége és automatizáció

Feladat VII.

- Egy fejlesztőcsapat automatizált regressziós tesztelést szeretne bevezetni egy banki mobilalkalmazásban. A cél az, hogy minden új frissítés után ellenőrizzék az alapvető funkciók működését.
- Elemezze a bevezetési kockázatokat, és tervezze meg az enyhítési stratégiákat!

A tesztautomatizálási megoldás fenntarthatósága

- A karbantarthatóság érdekében ajánlott a tiszta kód elveinek követése (Robert C. Martin: Clean Code):
  - egységes elnevezési konvenciók használata;
  - hardcode kerülése;
  - bemeneti paraméterek minimalizálása;
  - hosszú és bonyolult metódusok kerülése;
  - naplózás használata;
  - tervezési minták alkalmazása;
  - tesztelhetőség biztosítása.

A tesztautomatizálás megvalósítási, telepítési stratégiái (V. fejezet)

- Tesztszintek a CI/CD folyamatban
- Tesztelés integrációja a CI/CD folyamatba
- Tesztver konfiguráció
- API tesztautomatizálási függőségek

Tesztszintek a CI/CD folyamatban

- A felügyelet nélkül futtatható automata tesztek jól integrálhatók a CI/CD folyamatokba. A tesztszintek integrációi:
  - TAF/TAS konfigurációs tesztek: build során futnak, fájlok/elérési utak ellenőrzése;
  - Komponens tesztek: egyedi modulok, minőségbiztosítási kapuk;
  - Komponens integrációs tesztek: több modul együttműködésének ellenőrzése;
  - Rendszertesztek: teljes rendszer, gyakran a telepítési pipeline része;
  - Rendszerintegrációs tesztek: rendszerkomponensek együttműködése.

Tesztelés integrációja a CI/CD folyamatba

- Build vs Telepítés:
  - Tesztek (komponens, integrációs) a build fázisban
  - Telepítés csak sikeres build után
- Rendszertesztek beillesztése:
  - Telepítés közbeni tesztelés
    - Minőségkapu, de újrafuttatás = újratelepítés
  - Telepítés utáni tesztelés
    - Rugalmasabb, de nem állítja meg a hibás telepítést
- Egyéb célok:
  - Esti regressziós tesztelés és nem-funkcionális tesztek (pl. teljesítmény)

Feladat VIII.

- Egy e-kereskedelmi alkalmazásban a "Kosár" funkció felelős a termékek kosárba helyezéséért és azok mennyiségének frissítéséért.
  - A fejlesztői csapat CI/CD pipeline-t szeretne kiépíteni
- Határozzon meg tesztautomatizálási lehetőségeket a különböző tesztszinteken!

Tesztver konfiguráció

- A tesztver konfigurációkezelése kulcsfontosságú az automatizálás stabil működéséhez.
  - Tesztkörnyezet konfiguráció
    - URL-ek, hitelesítő adatok, közös könyvtárak használata
  - Tesztadat-kezelés
    - Kiadás-, környezet- és funkcióspecifikus adatok
  - Tesztkészlet-kezelés
    - Tesztkészletek funkció vagy cél alapján
    - Verzió és funkciókapcsoló alapján dinamikus futtatás
    - Teszteszközök és SUT együtt verziózva

API tesztautomatizálási függőségek

- API tesztautomatizálás kulcselemei
  - API kapcsolatok – üzleti logika, integrációk
  - API dokumentáció – paraméterek, fejlécek, válaszok
- Tesztelési szintek
  - Komponens- és rendszerintegráció
  - Szerződéses tesztelés
    - Interakciók és adatok egyeztetése
    - Korai hibaazonosítás
    - Fogyasztó által vezérelt vagy szolgáltató által vezérelt megközelítés

Tesztautomatizálási jelentések és mérőszámok (VI. fejezet)

- Adatgyűjtés forrásai
- Tesztvégrehajtás és naplózás
- Jelentéskészítés és vizualizáció
- Teszteredmények elemzése
- Tesztelőrehaladási jelentés készítése

Adatgyűjtés forrásai a tesztautomatizálásban

- Az adatgyűjtés lehetséges forrásai:
  - SUT naplók: Web/mobil UI, API, alkalmazások, szerverek, adatbázisok
  - TAF naplók: Teszteszköz audit- és hibainformációi
  - Build & deploy naplók
  - Éles rendszer naplók: teljesítménytrendek, éles figyelés
  - Képernyőképek, videók: manuális elemzéshez
  - Teszteszköz-bővítések: például kezdési/befejezési idő rögzítése
  - Teljesítménytesztek naplói: stressz, terhelés, csúcsterhelés

Tesztvégrehajtás és naplózás

- A tesztvégrehajtás során naplózandó:
  - Tesztesetek: kezdés/befejezés ideje, státusz
  - Tesztlépések, időzítések
  - Hibák: SUT vagy TAS eredetű?
  - Véletlenszerűség rögzítése (pl. random input)
  - Hibák újrajátszhatósága (pl. crash dump, stack trace)
  - Színes naplók → könnyebb elemzés

Jelentéskészítés és vizualizáció

- Az eredmények értékelése és kommunikálása:
  - Tesztjelentések → trendek elemzése
  - Korábbi futások összevetése
  - Tesztállapotok: Passed / Failed / TAS error / Inconclusive
  - Vizualizáció: diagramok, közlekedési lámpa ikonok
  - Harmadik fél eszközeinek integrálása (pl. táblázatok, riportálás)

Teszteredmények elemzése TAS/SUT alapján

- Tesztvégrehajtási adatok elemzése a hibák azonosításához
- A TAS adatok elsődlegesek, a SUT adatok másodlagosak
- Korábbi tesztfutások eredményeinek összehasonlítása
- Környezeti tényezők vizsgálata (pl. erőforrások, böngészők)
- Hibák visszakövetése: teszteset, lépés, log, képernyőkép
- Audit- és naplófájlok elemzése, korrelációs ID használata
- A hibaforrás lehet a SUT, a TAS vagy a környezet
- Hamis hibák elkerülése: infrastruktúra-problémák kizárása

Feladat IX.

- Egy mobilalkalmazás automatikus UI-tesztelésére készítettek tesztesetet Android emulátorral. A teszteset célja, hogy ellenőrizze a bejelentkezés utáni kezdőoldalon megjelenő felhasználónevet. A teszt azonban megbukik.
- Milyen adatokat vizsgálna meg annak megállapítására, hogy a teszteszközben (TAS) vagy a rendszerben (SUT) van a hiba? Soroljon fel 3-3 adatgyűjtési módszert!

Feladat X.

Az előző feladat adatgyűjtésének eredményét egy táblázatban foglalták össze. Elemezze az adatokat és vonjon le következtetést!

Oldal	Adatgyűjtési módszer	Adatgyűjtési eredmény
TAS	Tesztlog	Step 2: Assertion failed – "admin12" should be invalid
TAS	Screenshot	UI pozitív visszajelzést ad a „Felhasználónév elfogadva” szöveggel
TAS	Lokátor-definíciók	A validációs üzenet a megfelelő UI elemhez tartozik
TAS	Teszteset metaadatok	Valid username: csak kisbetű és szám, min. 8 karakter – de ez egy korábbi szabály!
TAS	Tesztparaméterek	A teszt az „admin12” nevet hibásnak várta – de a rendszer elfogadta
SUT	UI-interakció naplózás	Név beírása után „Valid username” visszajelzés jelenik meg
SUT	Rendszerlog (szerver oldali)	POST /validate-username → response: status=valid
SUT	Konzolnapló (frontend)	Validation rule applied: minimum 6 karakter, betű vagy szám
SUT	API válasz naplózás	API JSON válasz: { "valid": true }
SUT	Auditnapló (felhasználói események)	event=USERNAME_VALIDATED, input=admin12, result=success

Tesztelőrehaladási jelentés készítése, közzététele

- **Tesztelőrehaladási jelentés készítése**
  - Teszteredmények összegzése
  - SUT és környezet adatai
  - Hibaokok és felelősök rögzítése
  - Jelentéskészítő eszköz használata
  - Teszttrendek és regressziók vizsgálata
  - Dashboard és vizualizációk
  - Gépi tanulás támogatása
- **Közzététel és érintettek**
  - Webes, felhőalapú vagy e-mailes elérés
  - Jelentés tartalma szerepkör szerint változik
  - Célcsoportok: vezetők, operatív és technikai érintettek

A tesztautomatizálási megoldás ellenőrzése (VII. fejezet)

- A tesztautomatizálási környezet ellenőrzése
- Tesztautomatizálás: helyes működés és váratlan eredmények
- Statikus kódelemzés a tesztautomatizálásban

A tesztautomatizálási környezet ellenőrzése

- TAS telepítése és konfigurálása
  - Automatizált / manuális megoldások; Verziókezelés, repo frissítések
- Tesztkörnyezet ismételhetősége
  - Ugyanaz a TAS különböző környezetekben; Konfigurációk nyomon követése, dokumentálása
- Kapcsolódási ellenőrzések
  - Hozzáférés a szerverekhez, SUT-hoz; Jogosultságok és előfeltételek biztosítása
- TAF komponensek tesztelése
  - Funkcionális és nem-funkcionális ellenőrzések; Objektumkezelés, tesztnaplók, teljesítmény

Feladat X.

- A., Készítsen egy rövid ellenőrző listát, amellyel biztosítható, hogy a tesztautomatizálási környezet megfelelően működik a futtatás előtt.
- B., Írjon egy rövid tervet arról, hogyan biztosítaná, hogy a tesztautomatizálási eszközök telepítése és konfigurációja ellenőrizve legyen a projekt indulásakor.

Tesztautomatizálás: helyes működés és váratlan eredmények

- A tesztszkripteket, tesztcsomagokat ellenőrizni kell teljesség, helyesség és konzisztencia szempontjából. Az ellenőrzés lépései:
  - A tesztkészlet összetételének ellenőrzése
  - Új TAF funkciókhoz készített új tesztek ellenőrzése
  - A tesztek ismételhetőségének biztosítása
  - A teszteszközök beavatkozásának figyelembe vétele
- Tesztszkriptek hibái esetén gyökérok-elemzést kell végezni:
  - Elemzés; hibaforrások; módszerek.

Statikus kódelemzés a tesztautomatizálásban

- Hibák és sérülékenységek feltárása (SUT + TAF)
- Automatizált szkennelés → azonnali visszajelzés (SDLC korai szakaszában)
- DevSecOps része → proaktív hibafelismerés
- Hibák kategóriái: kritikus / magas / közepes / alacsony
- Segítség TAEs számára:
  - Minőségmérés, kód-komment javaslatok
  - Kódtervezés javítása (try/catch, ciklusok, erőforráskezelés)
  - Gyenge könyvtárhívások eltávolítása
- Példa: plaintext jelszavak a tesztszkriptekben → biztonsági kockázat

Folyamatos fejlesztés (VIII. fejezet)

- Teszteset javítási lehetőségek
- TAS műszaki elemzése, javítási lehetőségei
- Tesztver karbantartása
- A teszteszközök további felhasználhatóságai

Teszteset javítási lehetőségek

- A tesztesetek minősége és hatékonysága adatgyűjtés és elemzés által javítható. Megközelítései:
  - **Teszt hisztogram**
    - A grafikus megjelenítés segít azonosítani az instabil teszteseteket
    - Elemzésük alapján TAE dönthet a fejlesztésekről
  - **Mesterséges intelligencia**
    - AI, gépi tanulás és önjavító algoritmusok segíthetnek a tesztesetek karbantartásában
  - **Séma validáció**
    - Automatikus validálás (adattípusok, kötelező mezők) → rövidebb és hatékonyabb kód

Feladat XI.

- Egy vállalat automatizált regressziós tesztkészletet üzemeltet, amely több mint 600 tesztesetből áll. A teszteket minden éjjel futtatják, és a rendszer naplózza a futási eredményeket. Az utóbbi időben a fejlesztőcsapat azt tapasztalja, hogy:
  - az automatizált futások több órát vesznek igénybe;
  - sok teszt ismételten ugyanazokat az adatokat használja;
  - számos teszteset soha nem talál hibát, mégis naponta lefut;
  - néhány modulban a hibák gyakorisága magasabb, de ott kevés teszt van.
- Feladat:
  - Elemezze, hogyan lehet adatgyűjtéssel és elemzéssel azonosítani a tesztesetek fejlesztésének és optimalizálásának lehetőségeit, és fogalmazzon meg konkrét javaslatokat.